

Integrantes

- Luis Francisco Padilla – 23663
 - Jonathan Diaz – 23837
 - Antonio Merida – 201105
-

Demostración: Merge Sort es $O(n \log_2 n)$ usando el Método de Sustitución

Planteamiento del Problema

Queremos demostrar que el algoritmo *Merge Sort* tiene complejidad temporal $O(n \log_2 n)$ utilizando el **método de sustitución**.

La recurrencia que modela el tiempo de ejecución de *Merge Sort* es:

$$T(n) = 2T\left(\frac{n}{2}\right) + n, \quad T(1) = 1$$

Debemos probar que $T(n) = O(n \log_2 n)$.

Paso 1: Hipótesis de Inducción Inicial (Problema)

Intentamos primero con la hipótesis más natural:

Hipótesis inicial: Existe $c > 0$ tal que
 $T(n) \leq c n \log_2 n$ para todo $n \geq 1$.

Verificación del Caso Base ($n = 1$)

Para $n = 1$:

$$T(1) = 1$$

Según la hipótesis:

$$T(1) \leq c \cdot 1 \cdot \log_2 1 = c \cdot 1 \cdot 0 = 0$$

Esto implica:

$$1 \leq 0$$

Contradicción.

La hipótesis no funciona porque $\log_2 1 = 0$ hace que la cota sea cero, pero $T(1) = 1$.

Paso 2: Modificación de la Hipótesis de Inducción

Para resolver el problema del caso base, agregamos un término aditivo de orden menor.

Nueva hipótesis: Existe $c > 0$ tal que

$$T(n) \leq c n \log_2 n + n$$

para todo $n \geq 1$.

El término adicional n es de orden inferior a $n \log_2 n$ cuando $n \rightarrow \infty$, por lo que no afecta la clasificación asintótica $O(n \log_2 n)$.

Paso 3: Verificación del Caso Base con la Nueva Hipótesis

Para $n = 1$:

$$T(1) = 1$$

Evaluamos la hipótesis:

$$c \cdot 1 \cdot \log_2 1 + 1 = c \cdot 0 + 1 = 1$$

Por lo tanto:

$$T(1) = 1 \leq 1$$

El caso base se cumple para cualquier $c > 0$.

Paso 4: Paso Inductivo

Hipótesis Inductiva

Asumimos que para todo $k < n$ se cumple:

$$T(k) \leq c k \log_2 k + k$$

Demostración para n

Partimos de la recurrencia:

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

Aplicamos la hipótesis inductiva a $T\left(\frac{n}{2}\right)$:

$$T\left(\frac{n}{2}\right) \leq c \cdot \frac{n}{2} \cdot \log_2\left(\frac{n}{2}\right) + \frac{n}{2}$$

Sustituimos en la recurrencia:

$$T(n) \leq 2 \left[c \cdot \frac{n}{2} \cdot \log_2\left(\frac{n}{2}\right) + \frac{n}{2} \right] + n$$

Simplificamos:

$$T(n) \leq c n \log_2\left(\frac{n}{2}\right) + 2n$$

Paso 5: Simplificación del Logaritmo

Usamos la propiedad logarítmica:

$$\log_2\left(\frac{n}{2}\right) = \log_2 n - \log_2 2 = \log_2 n - 1$$

Sustituimos:

$$T(n) \leq cn(\log_2 n - 1) + 2n$$

Expandimos:

$$T(n) \leq cn \log_2 n - cn + 2n$$

Factorizamos:

$$T(n) \leq cn \log_2 n + n(2 - c)$$

Paso 6: Elección de la Constante c

Para que se cumpla:

$$T(n) \leq cn \log_2 n + n$$

necesitamos que:

$$cn \log_2 n + n(2 - c) \leq cn \log_2 n + n$$

Esto requiere:

$$n(2 - c) \leq n$$

Dividiendo entre $n > 0$:

$$2 - c \leq 1$$

Por lo tanto:

$$c \geq 2$$

Conclusión: Si elegimos $c \geq 2$, la desigualdad se cumple.

Paso 7: Conclusión Final

Hemos demostrado por inducción matemática que para $c \geq 2$ y todo $n \geq 1$:

$$T(n) \leq cn \log_2 n + n$$

Dado que el término n es de orden inferior a $n \log_2 n$, podemos escribir:

$$T(n) = O(n \log_2 n)$$

Por lo tanto, el algoritmo **Merge Sort** tiene complejidad temporal:

$$O(n \log_2 n)$$